

SECUREFLOW 2.0

AI-Powered DevSecOps Orchestration Platform

This preparation guide decomposes the SecureFlow project into clear, easy-to-explain concepts tailored for system design, security, and software engineering interviews. Use this document to walkthrough the architecture, problem space, system design choices, and common questions.

1. PROJECT OVERVIEW & AIM

SecureFlow is a developer-centric security platform that brings static application security testing (SAST), container vulnerability scanning, custom corporate policies, and dynamic application security testing (DAST) into a unified pipeline dashboard. It is designed to act as a central 'security command center' for developers.

Why SecureFlow Was Created

- * **Shift-Left Security:** Instead of checking for security bugs right before production release (which is slow and expensive), SecureFlow checks at every phase of the CI/CD pipeline, catching vulnerabilities as soon as code is pushed.
- * **Single Pane of Glass:** Unifies outputs from multiple disconnected CLI scanners (Gitleaks, Semgrep, Trivy, and OWASP ZAP) into one simple interface.
- * **Privacy-First AI:** Allows developers to ask an AI copilot (Void AI) how to fix bugs without exposing sensitive enterprise code to public cloud APIs.

Problems It Solves

- * **Security Tool Fragmentation:** Developers normally have to check separate logs or interfaces for secret leaks, code quality, and container bugs. SecureFlow unifies them into a single API and database.
- * **Slow Release Cycles:** Dynamic security scanning (DAST) can take 10+ minutes. If run synchronously, it blocks the main build line. SecureFlow solves this by running jobs asynchronously using back-end workers.
- * **AI Security & Privacy:** Large companies cannot send proprietary code to external APIs like OpenAI due to compliance. SecureFlow hosts local, secure LLMs on internal machines.

2. END-TO-END WORKING FLOW

SecureFlow relies on a strict state machine tracking 9 distinct stages in the pipeline. Here is exactly how a push travels through the system:

- * **Stage 1: Checkout:** Retrieves the repository code using Git.
- * **Stage 2: Code Scan (SAST):** Runs Gitleaks to find API keys/secrets and Semgrep to scan code structure for OWASP Top 10 vulnerabilities.
- * **Stage 3: Docker Build:** Builds the repository's container image to verify compilation and dependency structure.
- * **Stage 4: Container CVE (Trivy):** Scans the built Docker image for known security flaws (CVEs) in base system libraries and OS packages.
- * **Stage 5: Policy Gate:** The central engine evaluates all static findings against a YAML config ('policy.yaml'). It checks CVSS scores and rules, blocking or letting the pipeline pass.
- * **Stage 6: Deploy Staging:** Deploys the safe build to Google Cloud Run staging.
- * **Stage 7: DAST (OWASP ZAP):** The backend publishes a scan job to Celery. A worker pulls it from Redis, spins up a secure OWASP ZAP container, and dynamically tests staging endpoints.
- * **Stage 8: ZAP Policy Gate:** ZAP's dynamic findings are parsed and compared against the policy engine rules.
- * **Stage 9: Deploy Production:** If all checks pass, the backend triggers deployment to the live production environment.

Real-Time Frontend Updates

Every progress step or new security finding sends a webhook from GitHub Actions to the FastAPI backend. The backend processes it and immediately broadcasts it to the React Dashboard using WebSockets (backed by a Redis Pub/Sub backend). This ensures developers see pipeline updates live with a latency of less than 15ms.

3. KEY DIFFICULTIES & SOLUTIONS

Highlighting core architectural struggles and engineering solutions is crucial during system design interviews. Here are three main difficulties solved in this project:

Difficulty 1: Decoupling Long-Running Scans

OWASP ZAP scans can run for over 10 minutes, checking for cross-site scripting (XSS) and SQL injection. If the main FastAPI gateway ran these scans synchronously, it would block the HTTP workers, leading to client timeouts and a frozen dashboard.

Solution: Decoupled task distribution. The backend publishes a lightweight 'run_zap_scan' event to a Redis queue. An isolated Celery worker running on a separate machine pulls the task, invokes ZAP inside a container, and polls it. Once complete, it returns results to the backend via a secure callback API. The dashboard is kept responsive via WebSockets.

Difficulty 2: Normalizing Fragmented Security Schemas

Every security tool has a completely different output format. Semgrep returns line numbers and files, Trivy returns image layers and package names, and ZAP returns HTTP request paths and cookies.

Solution: A unified JSON ingestion engine on the backend. Created a normalized schema for findings. A parsing adapter transforms each scanner's output into a common interface: ScannerName, Severity, Title, Description, Filepath, Line, Remediation, and CVSS.

Difficulty 3: Secure Local RAG AI Integration

Providing accurate remediation steps using a local 3B parameter model (Qwen2.5) is difficult because the model has limited context. Additionally, we must ensure sensitive user information like access tokens never leak to the AI reasoning engine.

Solution: Retrieval-Augmented Generation (RAG) + Guardrails. Prompt injection filters and an automatic secret redaction utility sanitizes prompts. We ingest project policy configurations and current scan details into ChromaDB vectors. We use a fallback system that serves predefined solutions from the local SQL database if the AI model host goes offline, maintaining service availability.

4. TYPICAL INTERVIEW QUESTIONS

Q1: Why use Celery and Redis? Why not just use FastAPI background tasks?

Answer: FastAPI background tasks run within the same process memory space as the web server. If a background task runs intensive code or crashes, it can crash the web server or consume all available CPU cores. Celery decouples task execution to separate worker servers, ensuring high availability of the API server.

Q2: What happens if a Redis broker crashes during a DAST scan?

Answer: Celery tasks are stored in memory/disk by Redis. If Redis crashes without persistence enabled, pending scans would be lost. In production, Redis should run with AOF (Append Only File) enabled, and Celery tasks are configured to be 'acknowledged' only after completion, meaning the task would auto-retry when the system reboots.

Q3: How does the YAML Policy Engine work? How does it handle emergency overrides?

Answer: The Policy Engine reads a `policy.yaml` configuration. It extracts CVSS metrics, severity bounds, and exempts specific CVE IDs. If a team needs to hotfix a deployment, they can add the CVE to the allowlist with an expiration date. Once the expiration date passes, the build gate automatically resumes blocking that CVE.

Q4: Why did you build a local AI server rather than using standard cloud APIs?

Answer: Security audits and source code are highly confidential assets. Sending source files or internal secrets to public endpoints (like OpenAI or Gemini) violates strict compliance rules (SOC2, ISO 27001). Hosting Qwen2.5 locally on Ollama guarantees zero data leakage.

Q5: How do WebSockets handle reconnections or connection drops?

Answer: The React frontend implements automatic reconnection logic with exponential backoff. Upon reconnection, TanStack Query triggers a stale-while-revalidate invalidation, pulling the latest state from the database via REST APIs to resolve any updates missed during the disconnect.

Q6: How does the dashboard achieve low-latency updates?

Answer: The system utilizes Redis Pub/Sub. When GitHub Actions reports pipeline status, the backend publishes the event to a Redis channel. The WebSocket server instances subscribe to this channel and push updates instantly to connected clients (<15ms latency). This avoids heavy polling on the PostgreSQL database.

Q7: How does your database handle concurrency during active pipeline runs?

Answer: The backend uses async SQLAlchemy drivers (asyncpg/aiosqlite) to handle concurrent I/O operations without blocking execution threads. For SQLite development, it uses WAL (Write-Ahead Logging) mode to prevent write locks during concurrent scans.